MicroTraitLLM: Improvements

ECES 450/650

Singh,Kunvar <ks4268@drexel.edu>; Allipilli,Shanmukha Rao <sa3935@drexel.edu>; Dawud-Sulaiman,Abdullah <aod27@drexel.edu>; Patel,Chirayu <cnp68@drexel.edu>; Nhu,Chris <cn527@drexel.edu>; Rogers, Glen <gr467@drexel.edu>

Today's agenda

- Introduction
- Visual Question Answering (VQA)
- Fine-tuning
- Article Validation and Citation Accuracy (AVCA)
- Model Comparison and System Optimizations
- Conclusions

Introduction: MicroTraitLLM

Background

- Answers user question about microbes and microbial traits through academic article search and response.
  - Retrieval-Augmented Generation (RAG) system
  - Utilizes articles from the PubMed Open Access Subset (PMCOAS)
  - Provides both in-text citations and external citations.

Objective

- Improve the quality of MicroTraitLLM (MTLLM) responses through various projects
  - Individuals select interesting mini-projects to improve MTLLM
    - Studies, fine-tuning models, etc.
- Cumulatively integrate work into MicroTraitLLM through GitHub

Vision Question Answering: Objectives

1

Connect figures to questions

Let users ask natural-language questions directly about figures (plots, micrographs, tables) in microbiology papers.

2

Integrate with MicroTraitLLM pipeline

Plug VQA into the existing MicroTraitLLM pipeline so figure questions go through the same ingestion, routing, and answer-generation flow as text questions.

3

Evidence-grounded explanations

Return short explanations that explicitly reference visual evidence (axes, peaks, trends, or numbers) and can be surfaced as an ‘evidence’ block.

4

Prepare for quantitative evaluation

Design the VQA component so we can later benchmark accuracy on a small vision-QA dataset and compare models and prompts.

Vision Question Answering

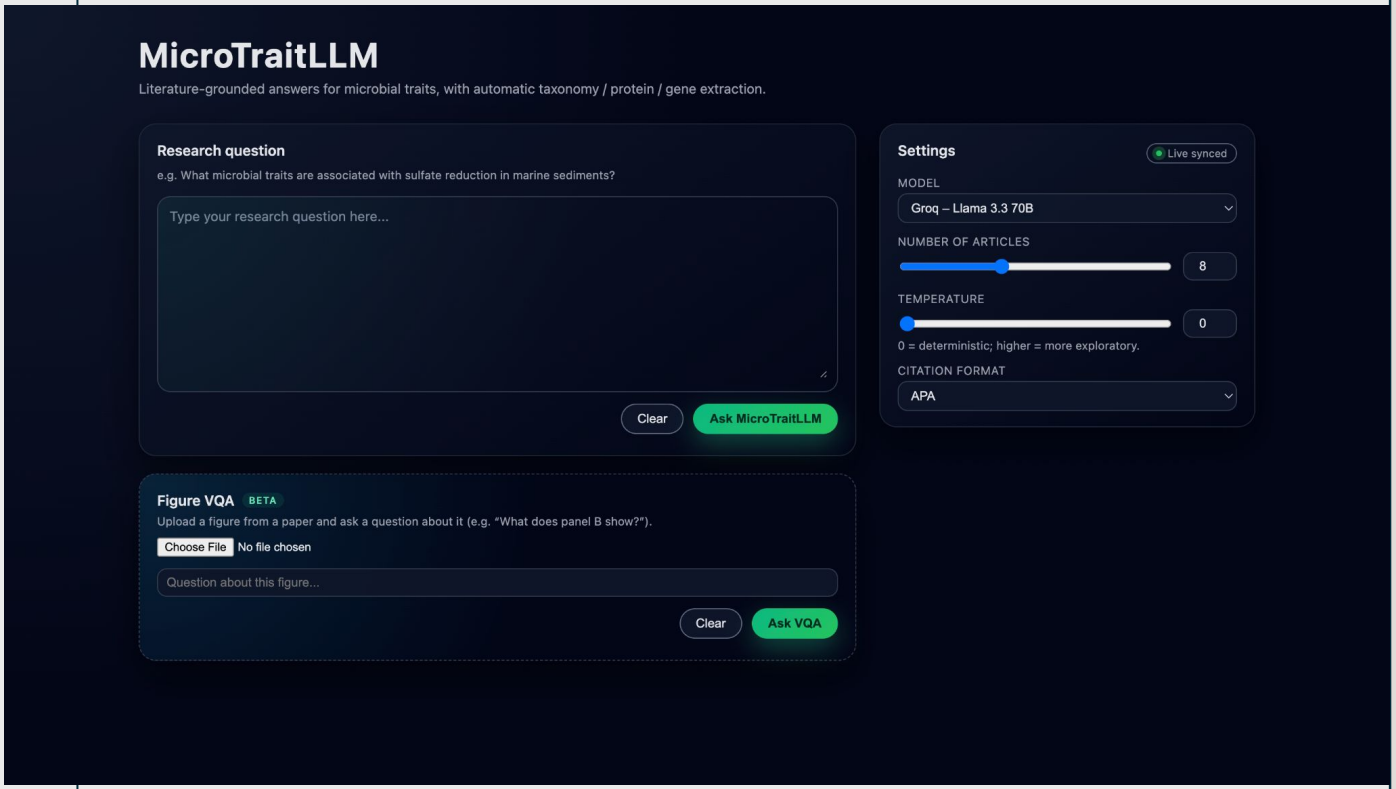
Integration

- New /vqa backend endpoint that accepts an image + natural-language question.
- vqa.py helper module that calls an OpenAI vision model, with fallback to a Groq vision model if OpenAI hits rate or credit limits.
- Front-end “Figure VQA” card in the MicroTraitLLM UI that lets users upload a figure and see the answer next to the text-based answer.
- Architecture designed to later consume figure metadata (caption, OCR, embeddings, chart data) from the ingestion/indexing pipeline.

Progress

- End-to-end VQA prototype running locally: curl and the UI both return answers for test figures.
- Successfully tested on a histogram figure; the system correctly described the axes, peak around intensity ~100, and overall dark image.
- Basic robustness: clear error messages for missing image/question and for API errors (rate limits, model deprecations).
- Code structured so figure_metadata can be added later without changing the UI.

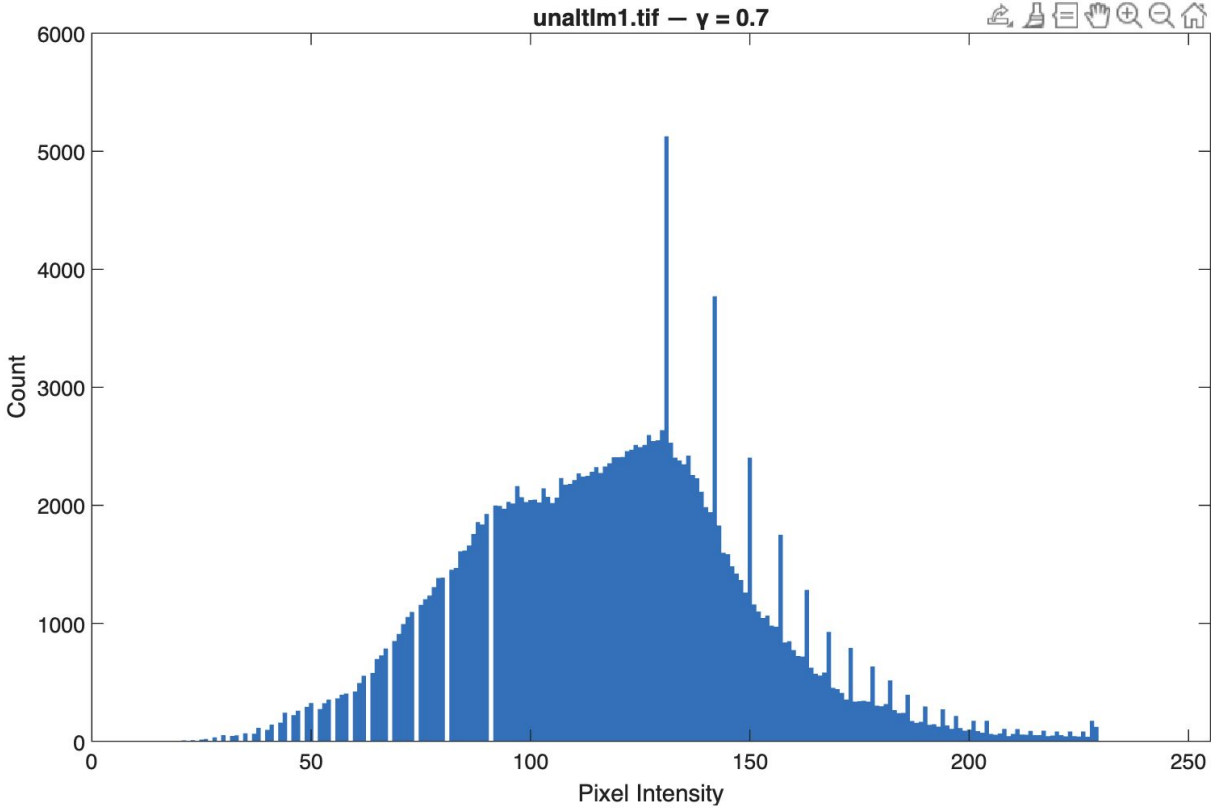
VQA: Results



Results

MicrotraitLLM with VQA integration UI

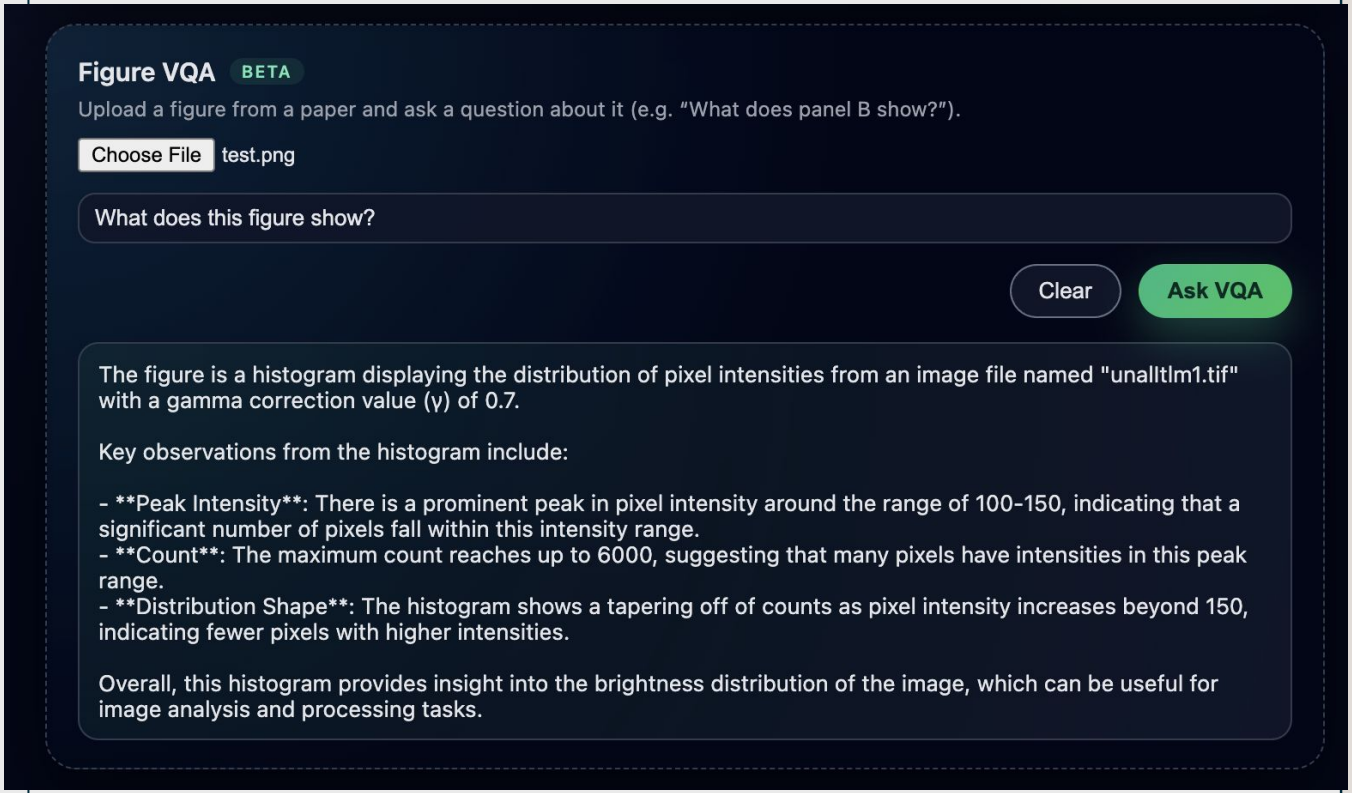
VQA: Results



Results

Histogram of pixel intensities for image *unalltm1.tif*

VQA: Results



Results

Histogram of pixel intensities for image *unalltm1.tif*

VQA: Results

Results

Histogram of pixel intensities for image *unalltm1.tif*

```
-->
abdullahdawud-sulaiman@Abdullahs-MacBook-Pro ~ % curl -X POST http://127.0.0.1:5000/vqa \
  -F "image=@/Users/abdullahdawud-sulaiman/Desktop/test.png" \
  -F "question=What does this figure show?"
{
  "answer": "The figure is a histogram displaying the distribution of pixel intensities from an image file named \"unalItlm1.tif\" with a gamma correction value (\u03b3) of 0.7. \n\nKey observations from the histogram include:\n\n- **Peak Intensity**: There is a prominent peak in pixel intensity around the range of 100-150.\n- **Count**: The maximum count of pixels at this intensity level reaches approximately 6000.\n- **Distribution Shape**: The distribution is right-skewed, indicating that there are fewer pixels with higher intensity values, as it tapers off towards the right side of the histogram.\n\nThis information can be useful for analyzing the brightness and contrast characteristics of the image."
}
abdullahdawud-sulaiman@Abdullahs-MacBook-Pro ~ %
```

Christopher Nhu

Fine-Tuning: Objectives

1

True Action Expert

Fine tuning a model to become a true expert in its respective domain beats out the current approach of manipulating outputs with system prompting

2

Affordability

By Fine tuning an open source model that can be downloaded locally, we reduce the need of using api tokens to access pay as you go models such as gpt 4o mini

3

Efficiency

Reducing the parameters of the model to a 7B parameter fine tuned model makes MTLLM more efficient, rather than using an untuned trillion parameter model such as 4o

4

Customizability

Through fine tuning, one has complete control over the behavior of the model, reducing hallucination rates and behaviors (such as accidentally creating a “references” section)

Fine-Tuning: Results

```
============================================================
Q6: What is the function of reverse transcriptase in retroviruses?
--------------------
FULL MODEL OUTPUT:
Retroviruses, such as HIV, utilize the enzyme reverse transcriptase to syn
thesize DNA from an RNA template, which is then integrated into the host c
ell's genome by the enzyme integrase (Baltimore, 1970).
--------------------
 > Entities Found: 3/3
 > Citations: Found!
 > No Footer: Clean!
============================================================
```

Figure 1: Example output from finetuned model

```
{
  "context_list": "1. Article: Legume-Rhizobia Symbiosis. Summary: Rhizobia are soil
  "question": "What is the role of leghemoglobin in legume root nodules?",
  "answer": "Leghemoglobin protects nitrogen-fixing bacteroids within root nodules f
},
```

Figure 2: Example training entry

Results Summary

Model responses are consistent, of an academic tone, provide citations, and no longer generate footers with citation links at the bottom of the response.

Fine-Tuning

Integration

To complete integration, I need to connect the model to Pubmed to prove it is able to take context from articles automatically.

Upload LoRA adapters for people to download along with Mistral to finetune

Progress

Fine tuned a 7B parameter model successfully on an RTX 4090

Created a training and testing set consisting of what the model might see when reading pubmed articles (question, context, answer)

Developed an evaluation script to evaluate model responses to look for key words and other parameters

Article Validation and Citation Accuracy

AVCA: Objectives

1

Article Validation

Build a scoring system that evaluates scientific articles based on recency, journal reputation, citation count, and topic relevance.

Support noisy or metadata-poor corpora using tolerant default thresholds.

Produce a standardized confidence label for each article (high / medium / low)

2

Topic Relevance Modeling

Implement an embedding-based similarity function (TF-IDF baseline; LLM embeddings later).

Quantify query–document alignment to filter out irrelevant sources.

Provide explainable numeric scores for debugging and threshold tuning.

3

Citation Accuracy Checking

Parse numeric citations in model-generated answers.

Extract PMCID/DOI identifiers from reference lists.

Match citations to retrieved articles and check snippet-to-article similarity.

4

System Integration

Produce a unified interface returning cleaned citations, validation scores, and error flags.

Ensure compatibility with RAG-based retrieval.

Each subsystem is modular and ready for integration into the larger pipeline.

Article Validation and Citation Accuracy

AVCA: Process

1

Data preparation

Started from a sample corpus: ~18,000 text files (~91 MB).

Drew a random subset of 3,000 documents for testing.

Parsed each file into an Article object:

- first line → title
- next few lines → pseudo-abstract
- simple metadata: journal = "SampleCorpus", year = 2020, citation_count = 0.

2

Embedding setup

Implemented a temporary embedding layer using TF-IDF (scikit-learn).

Fit the TF-IDF vectorizer on a subset of the corpus to capture vocabulary.

Patched the embed_text() function in the AVCA module so that:

- query text and article text are converted to TF-IDF vectors,
- cosine similarity gives a topic relevance score.

3

Article validation pipeline

For a query, ran validate_articles (query, articles) on the 3,000 document subset.

For each article, computed four scores: recency, journal reputation, citation count (all zero in this corpus), topic relevance.

Combined them into a composite validation score using fixed weights.

Applied a minimum threshold min_score_to_keep = 0.15 and kept all passing articles.

Collected the resulting scores to generate:

- the score histogram and
- the low/medium/high band counts shown on the Results slides.

4

Citation accuracy check

Took the top-scoring articles from the validation step.

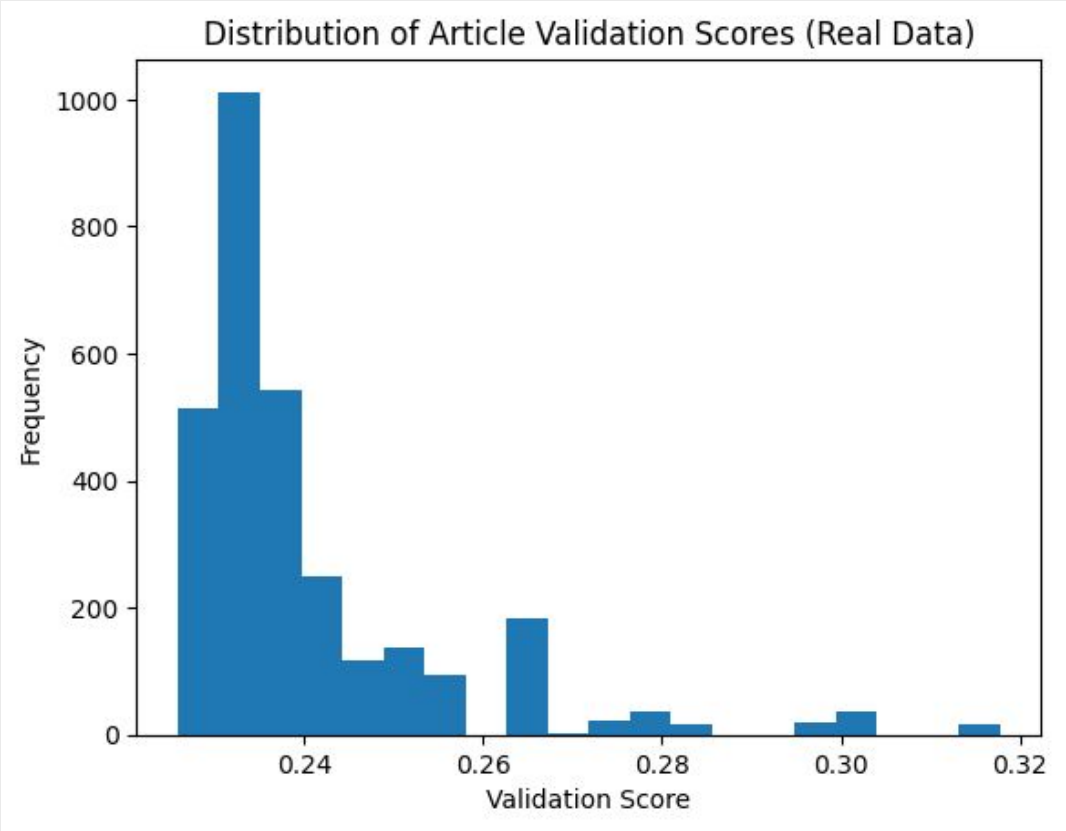
Constructed a short synthetic answer that cites them as [1], [2], [3].

Built a reference list with PMCID-style identifiers based on the filenames.

Ran check_citations() to:

- map numeric citations → identifiers → Article objects,
- compute snippet-to-article similarity,
- label each citation as valid, mismatch, or not_found.

AVCA: Results



Results

Successfully validated 3,000 real text documents from a 91 MB dataset (subset of 18k files).

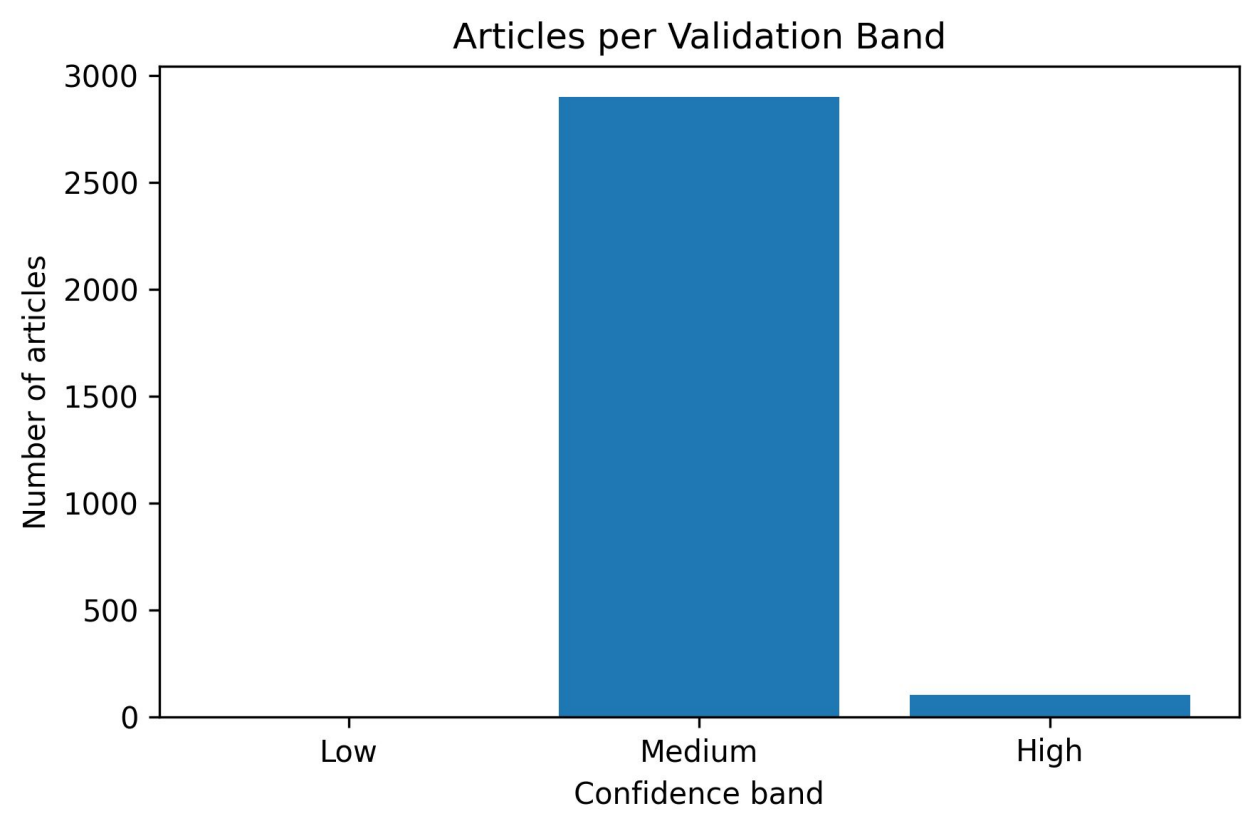
System generated a clear score distribution (0.15–0.34 range using TF-IDF embeddings).

Top-ranked documents contained relevant microbiology terminology, showing meaningful separation even with noisy input.

Scores fall into a fairly narrow band, but there is still visible variation driven by topic relevance

This confirms that the module runs at scale, produces interpretable outputs, and is ready for integration with retrieval and LLM subsystems.

AVCA: Results



Results

Citation checker mapped synthetic PMCID-style references to retrieved articles and produced per-citation diagnostic reports.

The end-to-end pipeline produced stable performance and no runtime errors, indicating readiness for integration with upstream RAG retrieval and downstream LLM augmentation.

Thresholding the scores into three bands (low / medium / high) gives:

0 low, 2,897 medium, 103 high.

This shows that with our current weights and TF-IDF embeddings, almost the entire corpus passes a minimum quality bar, while about 3–4% of documents are promoted to a high-confidence set for downstream use.

Article Validation and Citation Accuracy

Integration

Article Validation subsystem fully implemented, tested on real data, and producing interpretable scores.

Citation Accuracy subsystem functional with identifier extraction, snippet similarity, and hallucination removal.

All components modular and compliant with overall project structure.

Progress

Integrated real data evaluation: 3,000 samples processed successfully with full score computation and topic relevance modeling.

Added configurable thresholds and improved handling of unknown journals to support noisy datasets.

Implemented realistic testing workflow in Colab, including histogram generation, top-article inspection, and citation mapping validation.

Ready to plug into the central retrieval pipeline and evaluation harness.

Model Comparison: Objectives

1

BioGPT-FineTuned

Total Organisms Extracted: 15
Average Latency: 15.33s
Successful Extractions ratio: 15:15

2

biogpt

Total Organisms Extracted: 11
Average Latency: 14.80ms
Successful Extractions ratio: 11:15

3

gpt2-baseline

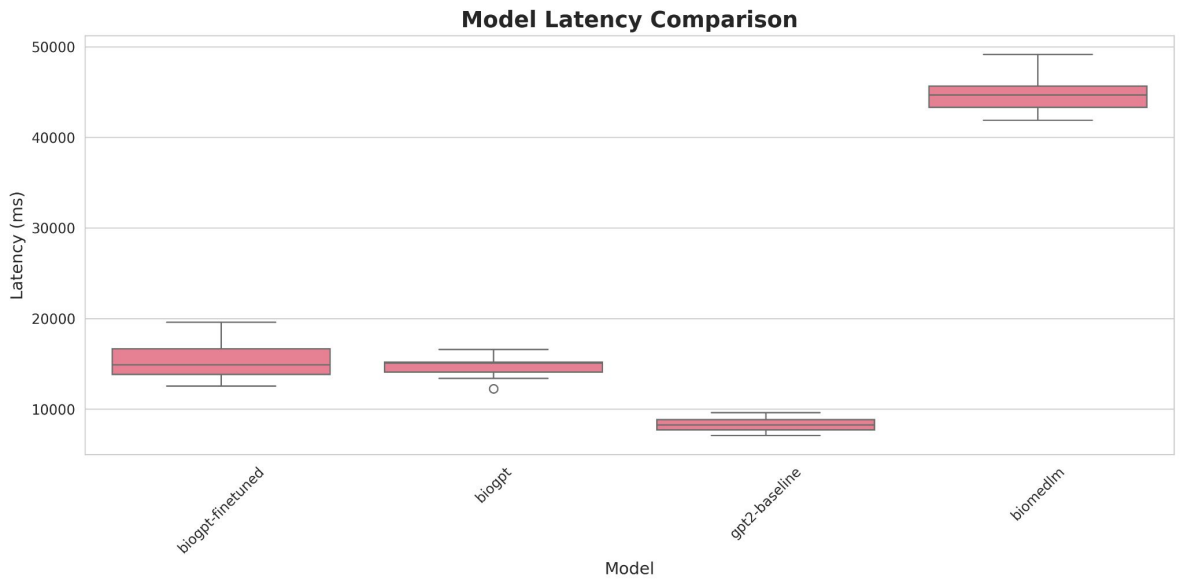
Total Organisms Extracted: 3
Average Latency: 82.78s
Successful Extractions ratio: 3:15

4

biomedlm

Total Organisms Extracted: 10
Average Latency: 44.97s
Successful Extractions ratio: 10:15

Latency Comparison



Model Latency

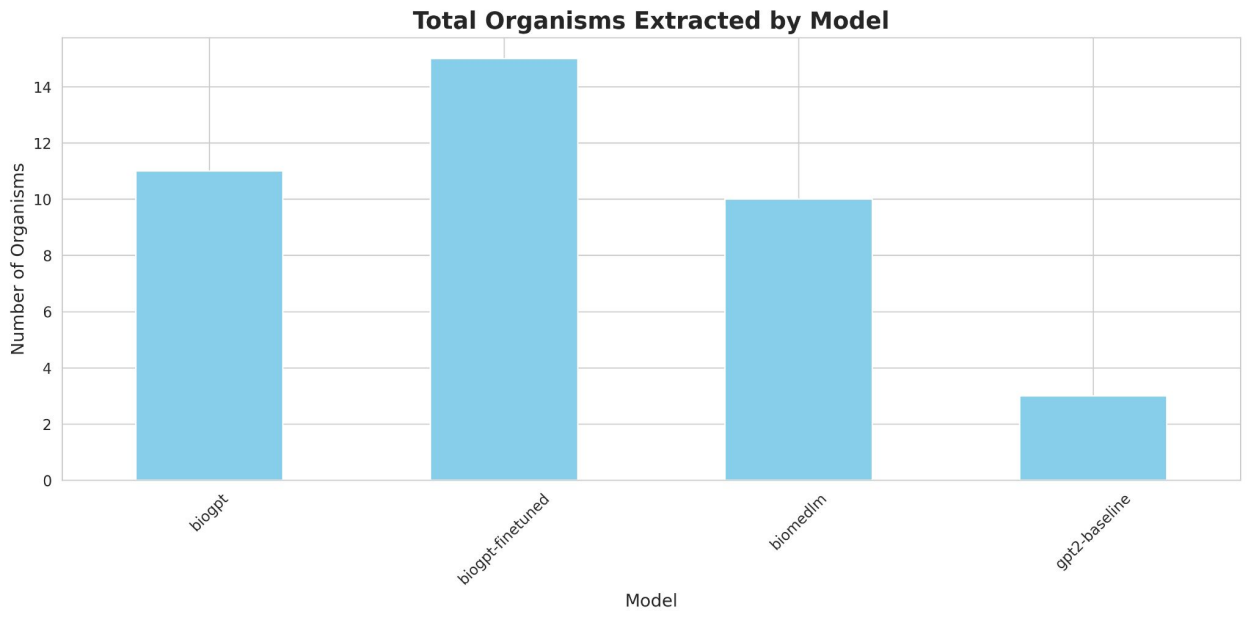
BiomedLM offers the fastest inference with median latency around 8,000ms and tight distribution (7,000-10,000ms range)

BioGPT and BioGPT-finetuned show similar performance at ~14,000ms median, with BioGPT displaying one outlier BioGPT-finetuned exhibits slightly better consistency than baseline BioGPT

BiomedLM shows 3-6x slower latency with median around 45,000ms, making it impractical for real-time applications despite high extraction performance

Speed-accuracy tradeoff: fastest model (GPT-2-baseline) extracts fewest traits; highest-performing model (BioGPT-finetuned) maintains reasonable latency

Organisms Extracted



Total Organisms extracted by each model

BioGPT-finetuned leads in organism identification with 15 organisms extracted

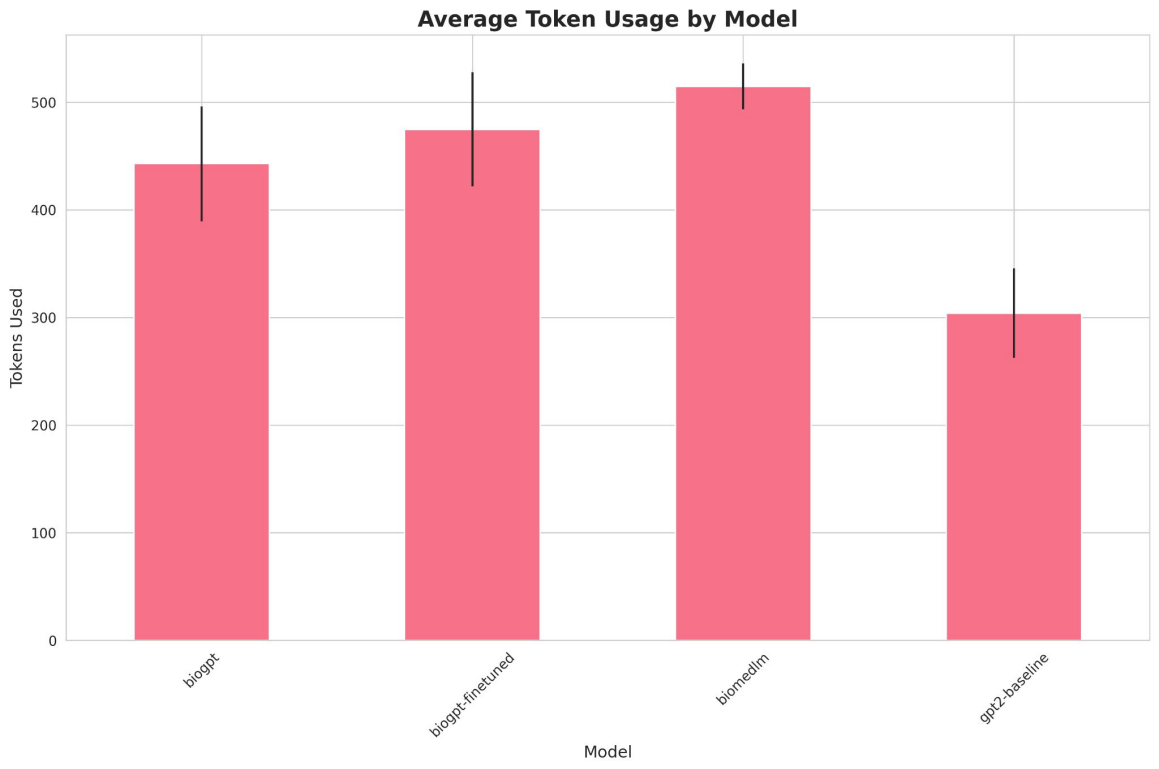
BioGPT baseline follows closely at 11 organisms, showing fine-tuning provides ~36% improvement

BiomedLM extracts 10 organisms, performing comparably to BioGPT baseline

GPT-2-baseline again underperforms with only 3 organisms, reinforcing need for specialized models

Organism extraction patterns mirror trait extraction results, suggesting consistent model capabilities across entity types

Token Usage



Average Token usage per model

BiomedLM uses most tokens (average ~515 tokens) with highest variability, indicating more verbose outputs

BioGPT-finetuned shows second-highest usage at ~475 tokens with moderate variance

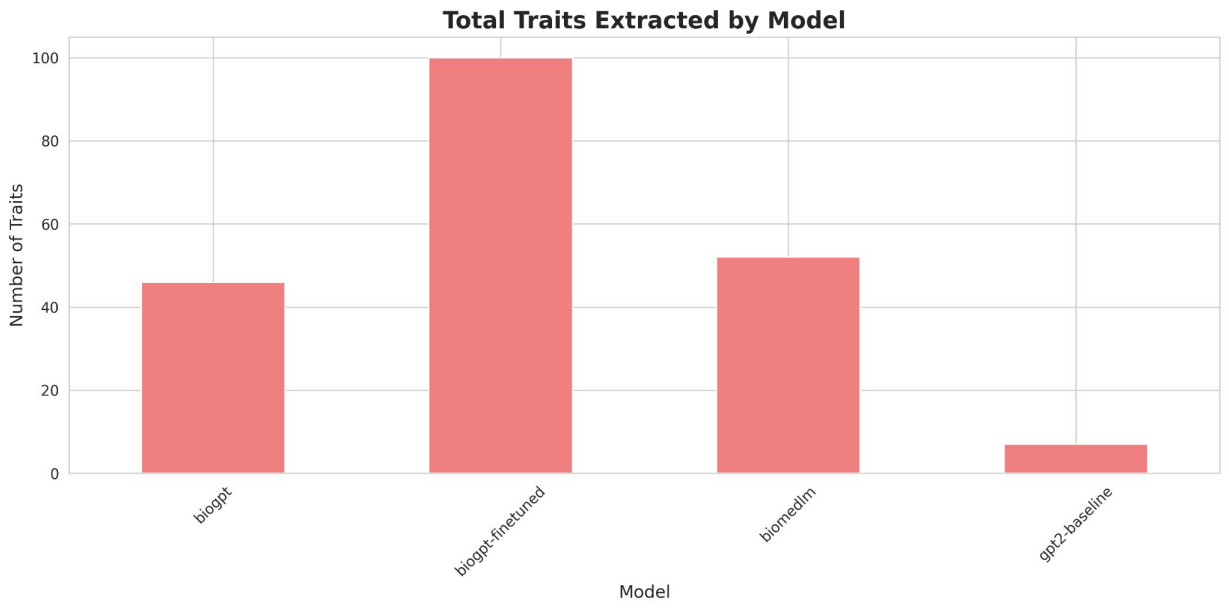
BioGPT baseline averages ~440 tokens with relatively consistent output length

GPT-2-baseline is most token-efficient at ~300 tokens but with high variance (260-350 range)

Token efficiency inversely correlates with extraction performance: lower token usage models extract fewer entities

BioGPT-finetuned balances extraction quality with reasonable token consumption

Traits by model



Total Traits extracted by model

BioGPT-finetuned demonstrates superior extraction capability with 100 traits extracted, significantly outperforming all other models

BiomedLM shows moderate performance at 52 traits, positioning it as the second-best option

BioGPT baseline achieves 46 traits, suggesting fine-tuning provides ~2x improvement

GPT-2-baseline severely underperforms with only 7 traits extracted, indicating domain-specific training is critical for this task

Fine-tuning on domain-specific data appears essential for trait extraction tasks

Model Comparison: Results

Fastest Model: gpt2-baseline (8278.21ms avg)
Most Organisms Extracted: biogpt-finetuned (15 total)
Most Traits Extracted: biogpt-finetuned (100 total)

- biogpt-finetuned: 0.0652 organisms/second
- biogpt: 0.0495 organisms/second
- gpt2-baseline: 0.0242 organisms/second
- biomedlm: 0.0148 organisms/second

Model Comparisons

Integration

Models Evaluated:

- BioGPT-FineTuned (domain-adapted)
- BioGPT (baseline)
- BiomedLM
- GPT-2-baseline

Integration Approach:

- Hugging Face Transformers pipeline
- Standardized prompt templates for consistency
- 15 biomedical queries across all models
- Automated metrics collection (latency, tokens, success rate)

Evaluation Metrics:

- Entity extraction accuracy (organisms & traits)
- Inference latency (ms)
- Token efficiency
- Success rate per query

Progress

Key Findings:

- BioGPT-FineTuned: Best overall performer (100% success, 100 traits, 15 organisms)
- Domain-specific fine-tuning provides 2x improvement over baseline
- GPT-2-baseline inadequate for biomedical tasks (20% success rate)

Performance Trade-offs:

- Speed: GPT-2 fastest, BiomedLM slowest (6x difference)
- Accuracy: Fine-tuned models extract 3-14x more entities
- Efficiency: Higher token usage correlates with better extraction

Recommendations:

- Deploy BioGPT-FineTuned for production (optimal accuracy-speed balance)
- BiomedLM unsuitable due to high latency
- Fine-tuning essential for specialized biomedical applications

Model conclusion

Recommendations

1. For speed: Use gpt2-baseline (fastest average latency)
2. For accuracy: Use biogpt-finetuned (most organisms extracted)

Performance Trade-offs

- biogpt-finetuned: 15 organisms @ 15326ms avg latency
- biogpt: 11 organisms @ 14801ms avg latency
- gpt2-baseline: 3 organisms @ 8278ms avg latency
- biomedlm: 10 organisms @ 44973ms avg latency

System Optimizations: Objectives

1

Local Database Hosting

Utilize the File Transfer Protocol (FTP) service provided by NCBI

Download relevant articles or complete article sets for use

2

Lightweight Reranker

Implement a lightweight reranker to validate the quality of articles

3

Top K Vector Ablation Study

Find the optimal top K vectors to compare a vectorized version of MTLLM to the standard

System Optimization

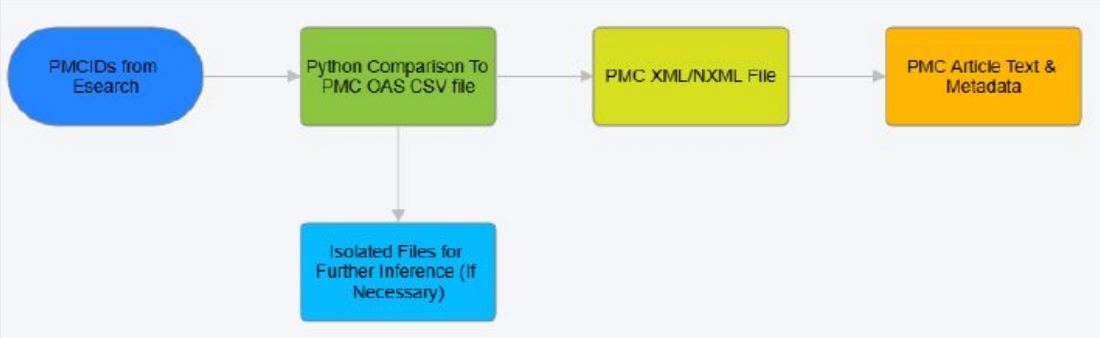
FTP System

- MTLLM can now downloads .tar.gz files containing all articles that are a part of the PMCOAS.
  - Baseline files
    - Larger archives containing most of the article data
    - Check all .tar.gz files to see if dates match with already downloaded files
    - If the dates don’t match, delete old files and download new ones
  - Incremental files
    - Smaller files for adjustments that happen between larger baseline updates
    - Updated daily
    - Check if incremental file is the same as the current day, update if not

Lightweight Reranker

- FlashRank selected as lightweight reranker
  - Ease of implementation
- Implementation was successful but potentially interfered with results from AVCA
  - Temporarily removed to avoid conflicts with answer reporting
  - When successful, FlashRank reranking generally aligned with previous iterations from MTLLM

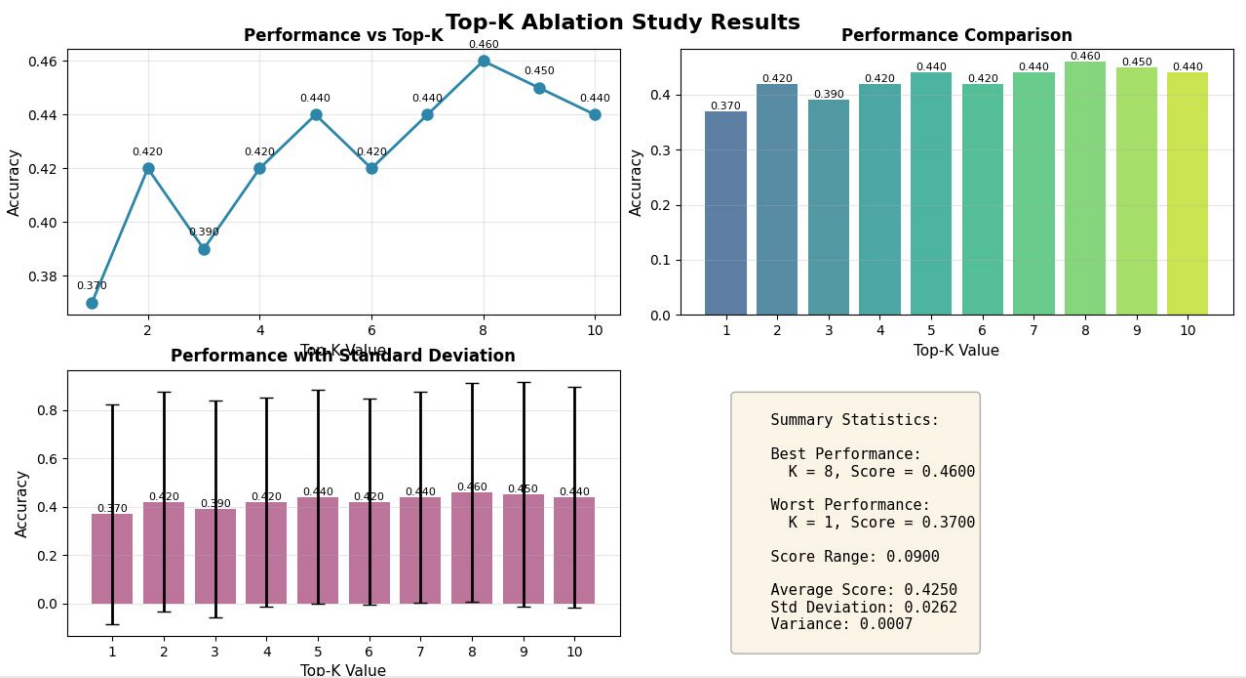
System Improvements: FTP Process



FTP Process

- Obtain PMCIDs like before
- Compare PMCIDs to CSV files containing metadata on .tar.gz archives
- Isolate text and metadata from the archive
- Store as a JSON file for future inference
  - Deleted after user session completes

System Improvements: Results



Top K Ablation Study

Top Vector: K = 8

Pattern trends can be attributed to varying scores between Gemma2:2b and Llama3.2

38000

Samples were used for testing across 20 questions

K = 8

Best number of vectors

Conclusions

Overall Project

- Successful initiation of various mini projects to improve MicroTraitLLM
  - Vision Question Answering backend shows potential for future use
  - New fine-tuned Mistral model presents new frontier for MTLLM usage
  - Answer validation and citation accuracy improved MTLLM future answer accuracy
  - Model comparisons demonstrate the validity of various other models, including those that are already fine-tuned on other biomedical literature
  - System optimizations create a more reliable system for future utilization

Future Work

- Exploring larger models (12 billion parameters and larger)
- Exploring additional datasets to utilize for inference
- Better front-end UI implementations

Thank you

If you have any questions, contact:
gr467@drexel.edu